

Flash-MaxSim 向昇腾 NPU 迁移的评估与实施建议

结论

值得迁移，优先迁移“分块计算、片上规约、不落盘完整相似度矩阵”的方法。 这是论文最有价值、也最不依赖 GPU 型号的部分。对昇腾 NPU，参数仍然重要，但论文中的 BLOCK_Q、BLOCK_D、BLOCK_K、warp 数、流水级数和调度阈值只是 A100/H100 上的实现选择，不能直接作为 NPU 的最优配置。

建议先实现 FP16/BF16 推理前向，再根据业务需求依次加入变长输入、训练反向、INT8 和超显存语料流式处理。论文中的 GPU 加速倍数不能作为 NPU 性能承诺；在 NPU 上最有把握保留的是**中间张量占用显存的数量级优势**。真正的延迟收益必须在目标芯片和真实输入分布上测量。

本文以昇腾 CANN/Ascend C 为主要迁移目标。若目标是其他 NPU，核心算法判断仍适用，但硬件映射和接口需要重做。

1. 论文到底优化了什么

MaxSim 对每个查询 token 找出最相似的文档 token，再把这些最大相似度相加：

$$\text{score}(Q,D)=\sum_{i=1}^{L_q}\max_{1\leq j\leq L_d}\langle Q_i,D_j\rangle.$$

朴素实现先生成形状为 [Nq, B, Lq, Ld] 的完整相似度张量 s，随后沿 Ld 做 max，沿 Lq 做 sum。论文的 Flash-MaxSim 将 Q、D 分块相乘，得到一个 [Bq, Bd] 子块便立刻规约，片上只保留每个查询 token 的当前最大值，最后输出每个查询—文档对的标量分数。训练前向额外保存获胜文档 token 的索引，反向依索引计算梯度，不构造完整的 ∇s。

以论文的图片页对图片页 ColPali 形状为例：B=10,000、Lq=Ld=1024、d=128。单是 FP16 的 s 就约为 21 GB，而 FP16 文档嵌入约为 2.62 GB；这解释了为什么去掉 s 比微调某个 tile 参数更关键。**这不是所有 ColPali 查询的默认形状**：短文本查询长文档更接近论文的 32×1024 或 128×1024 行，收益要单独测量。（论文 §3–5、表 18、表 22–23）

2. 哪些方法适合迁移，分别要修改什么

论文方法	迁移价值	在昇腾 NPU 上需要的修改或验证
分块矩阵乘 + 在线 max + 最后求和	最高，先做。 不写出完整 s 的数学结构与硬件无关；长 Lq、长 Ld、大候选集尤其受益。	用 Cube 执行 qb1k × Db1k [⊤] ，让结果进入 Vector 可处理的片上空间，做掩码、行 max、跨 D 块更新最大值及最终 sum。重点检查 Cube→Vector 的搬运与同步开销，确认中间 s 没有落到 Global Memory。Ascend C 的 Matmul/向量融合接口可表达此数据流，但 tile 排布必须按芯片重设。
FP32 累加与数值协议	必须保留。 论文将非量化路径视作与指定精度协议下的稠密计算数值等价，而非逐位相同。BF16 长序列尤其需要 FP32 规约。	分别确认乘法、矩阵乘累加、在线最大值和最终求和的实际 dtype；对照独立 FP32 基准检查分数误差、argmax 和 Top-K。处理相等最大值时应规定固定的选取规则，训练反向须与前向一致。
查询分块、文档分块和维度分块	高。 控制片上工作集，避免容量不足和无效重复加载。论文在 d>512 时引入 split-d。	在昇腾上重新选择 Bq/Bd/Bk、并行核数、数据布局、预取和双缓冲；根据 L1/L0/UB 容量、对齐和 Cube/Vector 吞吐确定，不照搬 GPU 的 d>512 阈值。小 d 和短序列要防止调用、转换及同步成本盖过收益。

论文方法	迁移价值	在昇腾 NPU 上需要的修改或验证
变长文档的紧凑存储和按长度寻址	高，前提是语料长度差异大。避免把所有文档补齐到最大 Ld。	用偏移量和长度数组描述 packed 文档；针对不规则长度处理尾块掩码、地址对齐和负载均衡。论文中的 cu_seqlens 是 CUDA 侧表示，迁移的是“紧凑存储 + 分段寻址”，不是接口名称。
反向仅保存 argmax，按获胜索引计算 $\nabla Q/\nabla D$	训练场景价值很高。避免完整 s 与 ∇s ；论文的主要训练收益是峰值内存与更大 batch 的可行性。	实现 ∇Q 的 gather 和 ∇D 的多源聚合。先用简单的正确实现建立基线，再比较原子累加、按目标 token 分组/CSR 两种方案；原子支持、竞争程度及构建 CSR 的成本都要在目标芯片实测。不要直接复制论文的 GPU 默认分支。
INT8 文档嵌入与延后反量化	有条件适合。对大语料可减少索引存储和读取；属于近似路径，不能视为非量化路径的“精确”替代。	验证目标芯片的 INT8 矩阵乘输入/输出 dtype；按 token 保存量化尺度，在点积后、max 前恢复尺度。需要同时测索引大小、端到端延迟、Top-K 和 nDCG；对短查询或小候选集可能没有速度收益。
超显存语料按块流入设备	适合超大语料。计算内核天然按文档独立打分。	按设备内存和主机—设备带宽选块大小，尽量重叠搬运与计算；单独报告主机搬运时间，不能把只测内核的结果当端到端吞吐。

论文还展示将相同的 hard-selection 思路用于 Chamfer distance。它说明**模式可以复用**，但不能据此推断任意带 max/min/argmax 的算子都能得到相同收益；应先确认原实现确实会产生大量、马上被规约掉的中间结果。（论文 §4、§6、附录 D/F/K/L）

3. NPU 迁移的关键差异

- 矩阵乘和规约之间的数据流。** 昇腾 Cube 侧使用 L0A/L0B/L0C 等存储，Vector 侧使用 UB。能否把矩阵子块高效送到 Vector 并在片上完成规约，直接决定“融合”是否真的节省全局内存流量。应使用目标产品支持的 Matmul 输出到 Vector 的路径，并检查编译结果和性能计数，而不是只看上层代码是否写在同一个算子中。如果某产品的实现必须把每个相似度子块经由 Global Memory 传给 Vector，仍可降低峰值内存，但论文最关键的相似度读写流量优势将明显减弱。
- 分块参数由硬件和形状共同决定。** Bq 太大，片上最大值与矩阵结果的工作集可能超预算；Bd 太小，Cube 利用率和搬运效率可能下降；Bk 则影响大 d 的复用与溢出。需要按芯片、精度和 Lq/Ld/d 形状分组调优。
- 训练反向的瓶颈可能变化。** GPU 上 CSR 与原子累加的时间/内存取舍不能直接外推到 NPU；应先看获胜索引分布、热点 token 的更新冲突，以及 CSR 构建占总时间的比例。
- 精度和布局是正确性要求。** 尾块要在 max 前置为 $-\infty$ ；量化尺度必须在比较最大值之前施加；矩阵乘与规约的精度、转置布局、padding 以及并列最大值的处理都须记录清楚。

不能直接照搬的项目： Triton/CUDA 内核代码、warp/SM 配置、TF32 参考路径、A100/H100 的最优 tile 与 $d>512$ 切换阈值，以及论文中的 GPU 延迟、加速倍数和最大候选集。论文的一个 query—document 对应一个计算程序的划分也只是起点；在 NPU 上可按候选文档和 query tile 提高并行度，但若拆开 query 维，还需要合并各 tile 的部分分数，并计入额外写回与调用开销。

昇腾官方文档明确说明了 [Cube/Vector 的存储与访问路径](#)、[融合算子的 Cube→Vector 数据流](#)、[Matmul 高阶接口](#)和 [ReduceMax 接口](#)。不同产品及 CANN 版本的能力和约束可能不同，实施前应锁定型号、版本并核对相应文档。

4. “思路更重要，还是参数更重要？”

对迁移是否值得做，思路更重要；对最后能跑多快，参数同样重要。

可以把它分成三个层次：

- 算法结构决定收益上限。** 不生成完整 s，把全局内存中的大张量变成片上小块和少量最大值状态，首先改变了内存占用及数据流量。单纯调 BLOCK_Q/BLOCK_D 无法让仍然写出完整 s 的实现获得这种数量级的显存收益。

2. **硬件映射决定收益能否兑现。** Cube 计算很快，但若每块结果都写回全局内存再做 Vector max，核心思路就被破坏。先设计正确的数据流和并行分工，再谈具体 tile 参数。
3. **参数决定局部最优。** 合适的 tile、布局、并行度和流水深度决定 Cube 利用率、片上占用、搬运与同步成本；错误参数甚至会让融合版慢于常规实现。参数必须在昇腾芯片上重新搜索，而不是移植论文的数字。

因此推荐的优先级是：**算法结构** → **NPU 数据流映射** → **精度正确性** → **参数调优**。这不是说参数不重要，而是参数只有在前两层成立后才有可调的高效实现。

5. 建议的验证顺序和判定标准

阶段	交付与验证	通过条件
0. 建基线	同一芯片上实现/记录 MatMul/einsum + max + sum、按文档分块的基线；固定精度与输入。	得到真实耗时、峰值显存、读写字节和正确性基准。
1. 前向原型	FP16/BF16、d=128，先覆盖 (Lq,Ld)=(32,300)、(32,1024)、(128,1024)、(1024,1024)。	不生成完整 s；分数误差符合预设阈值；Top-K 与 FP32 基准核对；大形状峰值内存主要随嵌入而非 Lq×Ld 增长。
2. 性能调优	按形状扫 Bq/Bd/Bk、并行度和缓存/流水配置；分别统计 Cube、Vector、搬运及同步。	与同精度、同候选集的基线比较延迟和峰值显存；记录小形状的回退条件。
3. 扩展能力	依据实际业务加入 packed 变长、训练反向、INT8、超显存流式处理。	每项单独核验数值质量与端到端性能；INT8 报告排序指标，训练报梯度误差和完整训练步内存。

论文的主要 GPU 实验在 A100/H100 上，训练峰值主要是 **MaxSim 算子层面的临时内存**，不包括编码器激活、优化器状态等全部训练内存。因此迁移评估也应同时报告“算子峰值”和“完整任务峰值”，不能把论文数值直接当作 NPU 整体系统收益。（论文 §5、Limitations、表 3）

来源

- Roi Pony 等，Flash-MaxSim: IO-Aware Fused Kernels for Late-Interaction Retrieval，arXiv:2605.29517v2，2026-06-17。本地原文：D:\temp\2605.29517v2.pdf。上述论文数字和方法以该版本为准。
- 昇腾 CANN 8.5.0 官方文档：上文链接的硬件架构、融合算子与 Ascend C API。具体芯片和软件版本应以实际部署环境为准。